Lab : Utilisation : Terraform & Docker

Le concept de l'**Infrastructure as Code** a pour but d'automatiser le déploiement des systèmes et infrastructures par l'utilisation de méthodes qui se rapprochent du développement logiciel, comme l'utilisation de scripts ou de fichiers de configuration. Ainsi, le processus de déploiement se trouve plus rapide, sans risque d'erreur humaine, répétable et au final mieux maîtrisé.

Les outils de type **Puppet** ou **Ansible** sont capables dans une certaine mesure d'interagir avec une couche infrastructure, toutefois leur rôle est privilégié pour ce qui est de la configuration des systèmes. À l'inverse, **Terraform** ne se préoccupe pas des systèmes d'exploitation, mais agit comme un formidable orchestrateur vers différentes solutions, à partir du moment où elles disposent d'une API. Leur utilisation conjointe est donc tout à fait possible, si ce n'est recommandé.

Dans le cadre d'une utilisation avec **Docker**, Terraform ne se préoccupe pas de la gestion des images. Leur création avec un **DockerFile** et leur publication sur une *registry* restent d'actualité et ceci est géré en dehors de Terraform. Son rôle démarre au moment de l'instanciation des conteneurs depuis une image. **HashiCorp**, l'éditeur derrière Terraform dispose toutefois d'un outil pour la gestion des images dont la logique est similaire : **Packer**.

1. Instancier un premier conteneur

1.1 Prérequis

Si le *runtime* Docker n'est pas présent sur votre machine, l'installation est très simple. Pour une **CentOS** ou une **RedHat**, il faudra en plus ajouter le dépôt Docker-ce à votre système.

```
$ sudo apt install docker.io
$ sudo systemctl enable docker.service
$ sudo systemctl start docker.service
$ sudo usermod -aG docker $USER
```

Terraform est distribué sous la forme d'un unique binaire compilé en statique, par conséquent il suffit d'extraire le contenu du ZIP téléchargé sur le site de l'éditeur et de le placer dans un répertoire, de préférence présent dans votre **${PATH}**.

```
$ wget
https://releases.hashicorp.com/terraform/0.12.24/terraform_0.12.24_linux_amd64.zip
$ sudo unzip -d /usr/local/bin/ terraform_0.12.24_linux_amd64.zip
Archive: terraform_0.12.24_linux_amd64.zip
  inflating: /usr/local/bin/terraform
```

Voilà, c'est toute la simplicité et l'élégance de Terraform. Il n'y a rien de plus à déployer ou configurer pour rendre l'outil fonctionnel. On peut d'ores et déjà écrire nos premières lignes de code.

1.2 Un conteneur en quelques lignes

Nous allons dans un premier temps instancier un conteneur avec la dernière image **Ubuntu** via un fichier Terraform. En ligne de commande, ce sera l'équivalent de ces deux commandes :

```
$ docker pull ubuntu:latest
$ docker run -d --name terraform_container ubuntu:latest
```

Il n'y a pas d'impératifs forts pour l'organisation des fichiers Terraform. Cependant, la convention veut que le code principal se trouve dans un fichier nommé **main.tf**.

Tout démarre par la déclaration du *provider*, en l'occurrence Docker dans notre cas, avec la méthode de connexion. Dans le cas le plus simple, il s'agit d'utiliser la *socket* UNIX locale, il faut donc avoir les droits sur la *socket*. Dans le cadre d'une utilisation à distance, avec SSH par exemple, cela amène donc à la nécessité de déclarer les *credentials* à utiliser pour s'authentifier. Toujours dans notre fichier **main.tf**, on déclare ensuite les ressources que nous souhaitons instancier. Pour un conteneur nommé **terraform_container** depuis la dernière image Ubuntu, cela donne :

```
provider "docker" {
  host = "unix:///var/run/docker.sock"
}

resource "docker_image" "ubuntu" {
  name = "ubuntu:latest"
}

resource "docker_container" "terraform_container" {
  image = docker_image.ubuntu.latest
  name  = "terraform_container"
}

must_run = true
 command = [ "tail",
    "-f",
    "/dev/null"

    ]
  }
```

1.3 Cycle de vie

Le conteneur est décrit dans un exemple de code, mais n'est pas encore instancié tant que l'on n'a pas fait appel à Terraform. Nous allons dans un premier temps avoir besoin du *provider* Terraform pour Docker **[1]**, le binaire que nous avons téléchargé n'en disposant pas. Terraform va automatiquement détecter le *backend* dont il a besoin depuis le code :

```
$ terraform init

Initializing the backend...

Initializing provider plugins...
- Checking for available provider plugins...
- Downloading plugin for provider "docker" (terraform-providers/docker) 2.7.0...

[...]

* provider.docker: version = "~> 2.7"

Terraform has been successfully initialized!
```

Nous allons maintenant générer un « plan ». Le plan ne crée aucun changement sur l'infrastructure, mais va analyser les actions qui doivent être entreprises pour appliquer les changements souhaités sur l'infrastructure. Cela permet de visualiser les actions que Terraform souhaite entreprendre avant qu'elles ne soient appliquées, en particulier lorsque ces changements sont sensibles. Optionnellement, le plan peut être sauvegardé dans un fichier.

```
$ terraform plan
Refreshing Terraform state in-memory prior to plan...
The refreshed state will be used to calculate this plan, but will not be persisted to local or
remote state storage.

docker_image.ubuntu: Refreshing state...
[id=sha256:4e5021d210f65ebe915670c7089120120bc0a303b90208592851708c1b8c04
bdubuntu:latest]

------------------------------------------------------------------------

An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # docker_container.terraform_container will be created
  + resource "docker_container" "terraform_container" {
    + attach          = false
    + bridge          = (known after apply)
    + command         = (known after apply)
    + container_logs  = (known after apply)
```

```
    + entrypoint       = (known after apply)
    + env              = (known after apply)
    + exit_code        = (known after apply)
    + gateway          = (known after apply)
    + hostname         = (known after apply)
    + id               = (known after apply)
    + image            =
"sha256:4e5021d210f65ebe915670c7089120120bc0a303b90208592851708c1b8c04bd"
    + ip_address       = (known after apply)
    + ip_prefix_length = (known after apply)
    + ipc_mode         = (known after apply)
    + log_driver       = "json-file"
    + logs             = false
    + must_run         = true
    + name             = "terraform_container"
    + network_data     = (known after apply)
    + read_only        = false
    + restart          = "no"
    + rm               = false
    + shm_size         = (known after apply)
    + start            = true

    + labels {
        + label = (known after apply)
        + value = (known after apply)
      }
  }

Plan: 1 to add, 0 to change, 0 to destroy.[...]
```

Si la revue du plan est conforme aux actions souhaitées, il peut être appliqué :

```
$ terraform apply
docker_image.ubuntu: Refreshing state...
[id=sha256:4e5021d210f65ebe915670c7089120120bc0a303b90208592851708c1b8c04
bdubuntu:latest]

[...]

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
[...]

docker_container.terraform_container: Creating...
docker_container.terraform_container: Creation complete after 0s
[id=2e6dc1f72de2ba701bbebb355d23515ff3380ff698d4dc39150bc8b8c4686ba3]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Vérifions que notre conteneur est bien instancié :

```
$ docker ps -a
```

```
CONTAINER
ID    IMAGE          COMMAND       CREATED          STATUS                PORTS   NAM
ES
133caea16805    4e5021d210f6   "/bin/bash"   12 minutes ago   Exited (0) 5 seconds
ago          terraform_container
```

C'est parfait, Terraform a fait le job. À l'issue de son exécution, un fichier **tfstate** a été généré afin de pouvoir conserver un lien entre ce qui a été généré et le code de l'infrastructure. Cela permet également de gérer l'ensemble du cycle de vie du *provisionning* de l'infrastructure, ce que nous allons faire en décommissionnant notre conteneur :

```
$ terraform destroy
docker_image.ubuntu: Refreshing state...
[id=sha256:4e5021d210f65ebe915670c7089120120bc0a303b90208592851708c1b8c04
bdubuntu:latest]
docker_container.terraform_container: Refreshing state...
[id=133caea16805a38a3997f7a2fad438e1030a6c3a09ec732db76e706dc22ec217]

An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:
# docker_image.ubuntu will be destroyed
- resource "docker_image" "ubuntu" {
    - id     =
"sha256:4e5021d210f65ebe915670c7089120120bc0a303b90208592851708c1b8c04bd
ubuntu:latest" -> null
    - latest =
"sha256:4e5021d210f65ebe915670c7089120120bc0a303b90208592851708c1b8c04bd"
-> null
    - name   = "ubuntu:latest" -> null
  }

Plan: 0 to add, 0 to change, 1 to destroy.[...]
```

Que ce soit utilisé avec Docker, **VMWare** ou **AWS [2]**, cette logique d'utilisation reste dans tous les cas valables.

2. Terraform dans le monde réel

L'exemple précédent n'était pas pertinent dans le périmètre d'un unique conteneur, mais permettait de s'approprier Terraform. Dès qu'il est question d'avoir plusieurs conteneurs qui doivent communiquer, d'exposer une application sur le réseau ou de permettre à un conteneur de disposer d'un stockage persistant, le besoin est tout autre.

L'outil de prédilection avec Docker est plutôt **Docker Compose**, mais d'une part l'utilisation du **YAML** rebute certaines personnes et d'autre part, son utilisation est

limitée à ce système de conteneurisation. Terraform propose une autre syntaxe pour arriver à ses fins.

Pour que le lecteur puisse facilement comparer, je suis parti de la *stack* **Docker Wordpress** sur le **dockerhub [3]** et je vous propose une version écrite en Terraform. Cette *stack* déploie un conteneur de base de données et deux volumes persistants. J'ai juste remplacé **MySQL** par **MariaDB** pour ne pas déployer un produit issu d'**Oracle** sur ma machine.

Voici donc ce que donne notre *stack* Wordpress réécrite en Terraform :

```
provider "docker" {
  host = "unix:///var/run/docker.sock"
}

resource "docker_network" "private_network" {
  name = "wp_net"
}

resource "docker_volume" "wp_vol_db" {
  name = "wp_vol_db"
}

resource "docker_volume" "wp_vol_html" {
  name = "wp_vol_html"
}

resource "docker_container" "db" {
  name  = "db"
  image = "mariadb"
  restart = "always"
  network_mode = "wp_net"
  mounts {
    type = "volume"
    target = "/var/lib/mysql"
    source = "wp_vol_db"
```

```
  }
  env = [
    "MYSQL_ROOT_PASSWORD=rootpassword",
    "MYSQL_DATABASE=wordpress",
    "MYSQL_USER=exampleuser",
    "MYSQL_PASSWORD=examplepass"
  ]
}

resource "docker_container" "wordpress" {
  name  = "wordpress"
  image = "wordpress:latest"
  restart = "always"
  network_mode = "wp_net"
  env = [
   "WORDPRESS_DB_HOST=db",
   "WORDPRESS_DB_USER=exampleuser",
   "WORDPRESS_DB_PASSWORD=examplepass",
   "WORDPRESS_DB_NAME=wordpress"

  ]
  ports {
   internal = "80"
   external = "8080"
  }
  mounts {
   type = "volume"
   target = "/var/www/html"
   source = "wp_vol_html"
  }
}
```

Un **terraform apply** plus loin, nous pouvons vérifier le déploiement de nos deux conteneurs et accéder à notre application sur le port **8080** :

```
$ docker ps
CONTAINER
ID  IMAGE           COMMAND                  CREATED        STATUS        PORTS
   NAMES
d52385968da1  mariadb          "docker-entrypoint.s…"  11 seconds ago   Up 10
seconds  3306/tcp           db
58b40a7d5d41  wordpress:latest  "docker-entrypoint.s…"  22 seconds ago   Up 21
seconds  0.0.0.0:8080->80/t  wordpress
```

Conclusion

La grande force de Terraform à mon sens est de ne pas jouer le rôle d'une couche d'abstraction aux différents *providers*. C'est souvent un facteur de bugs ou une limitation sur un dénominateur commun de fonctionnalités, qui s'en trouvent forcément réduites. En effet, les ressources manipulées sont directement celles de la solution administrée. On ne manipule pas une VM, mais une instance EC2 ou une machine virtuelle VM avec leurs spécificités et fonctionnalités, ne limitant donc pas l'éventail de possibilités à un sous-ensemble de fonctionnalités partagées.

Le grand intérêt de Terraform réside dans la capacité de pouvoir utiliser un même outil et un même flux de travail avec une grande variété de *providers*.